

Compliance of SSE Endpoint Implementation with SSE Specification

The SSE (Server-Sent Events) endpoint in the MCP server **appears to fully comply with the SSE specification** expected by modern browsers. Each aspect of the implementation matches standard requirements for EventSource. Below is a breakdown confirming each point:

- **Content-Type** `text/event-stream`: The server sends the SSE response with the HTTP header `Content-Type: text/event-stream`, as required for all SSE connections ¹. In the code, this is achieved by using FastAPI's `StreamingResponse` with `media_type="text/event-stream"`, which ensures the MIME type is correct. This tells the client (browser) to treat the response as an event stream.
- **Proper SSE event framing**: Events are sent in the correct format with the appropriate framing. Each message is formatted as one or more lines beginning with field prefixes and ending with a blank line. The code constructs each event frame with an `event:` line (specifying the event name) and a `data:` line (with the JSON payload), followed by **two newline characters** (`\n\n`) to delimit the end of the event ² ³. For example, the implementation creates frames like `"event: message\ndata: {...}\n\n"`. This satisfies the SSE protocol requirement that each event is separated by a double newline (a blank line), and uses the proper `event:` / `data:` fields for the EventSource to parse ³. (The server doesn't include an `id` or `retry` field, which are optional in SSE.)
- **Initial headers (no caching, keep-alive)**: The SSE endpoint sends the correct headers to establish a persistent, non-cached stream. Notably, it sets `Cache-Control: no-cache` to prevent any caching of the stream, and `Connection: keep-alive` to signal that the connection should remain open ⁴. These headers are standard for SSE responses – they ensure the browser or any proxy will not buffer the events and will keep the connection alive for continuous data. (The response is streamed without a `Content-Length`, using chunked transfer, which is also appropriate for SSE.)
- **Keep-alive heartbeats**: Yes – the implementation includes periodic heartbeat comments to keep the connection alive during idle periods. According to the SSE spec, a line starting with a colon (`:`) is a comment that the client ignores, often used as a heartbeat ³. The MCP server's SSE code takes advantage of this: if no new events have been published for a while, the server emits a comment line like `": keep-alive"` followed by a blank line (every 15 seconds of inactivity). These dummy events have no data and are ignored by the browser, but they prevent the connection from timing out ³. This is exactly the recommended practice for SSE: sending an empty comment periodically to maintain the open connection.
- **Standard EventSource compatibility**: The endpoint is designed to be consumed by the browser's **EventSource** API without any special tricks. A standard `EventSource` client initiates a connection with `Accept: text/event-stream`, which the server explicitly checks for on the `/mcp` GET route. If that header is present, it upgrades to the SSE stream (otherwise returning 404 for a normal GET) – this handshake is the normal pattern for EventSource ⁵. Once

connected, the server pushes events that follow the spec, so the browser will dispatch them correctly. For instance, the server sends some events with a custom `event: ready` field; the EventSource will dispatch those to any JavaScript listener for the `"ready"` event, and any events without an `event:` field would default to dispatch via the `message` event handler ⁶. In short, because the content type and framing are correct, **any modern browser's EventSource can consume this stream** normally (listening via `.onmessage` or `addEventListener` for specific event types).

- **No long-polling or hacks:** The implementation uses a true streaming response rather than relying on long-polling or other legacy comet-style techniques. Once the SSE connection is established, it stays open indefinitely and events are written to the client as they become available. There is a continuous loop on the server that only ends when the client disconnects, meaning **it does not close the HTTP response after each event**. This behavior is exactly what SSE is meant for – a single persistent HTTP connection kept open “forever” to push events ⁷. There are no workarounds like frequent reconnections or hidden iframes; it's a straightforward, modern SSE implementation using the EventSource protocol as intended.

Each of the above points is reflected in the provided code (specifically in `sse_routes.py` and the `SSEBus` logic). Therefore, we can conclude that **the SSE endpoint in the MCP server conforms to the SSE specification** and should work with any modern browser EventSource. The server sets the correct headers, formats events properly (with `data:` and optional `event:` fields, separated by `\n\n`), sends periodic keep-alive pings, and uses a persistent connection – no deprecated long-polling or streaming hacks required. Overall, it is fully aligned with what the EventSource API expects ¹ ⁴.

¹ ³ ⁶ Using server-sent events - Web APIs | MDN

https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

² ⁴ ⁵ The Power of Server-Sent Events(SSE) : Why SSE Are Your App's New Best Friend

<https://community.ibm.com/community/user/blogs/abishek-s/2024/11/28/the-power-of-server-sent-events-sse>

⁷ Replace long polling to receive events with Server Sent Events (SSE) · Issue #475 · matrix-org/matrix-spec · GitHub

<https://github.com/matrix-org/matrix-spec/issues/475>